Universidad de los Andes
Department of Systems and Computing Engineering

ISIS-2111 PLE
Project 1 Review Activity
Date: August 29, 2026

CosmosLog Mission

Design and Validation of a Review Activity for the
Objectilang Grammar Assignment

Prepared by Carla Gonzalez Mina

This report documents the design, the teaching materials, and the technical validation of a ninety minute review session built for ISIS-2111 Programming Language Essentials. The session prepares students for Task 1 of Project 1, writing the EBNF grammar of Objectilang, by having them run the same procedure end to end on a different, smaller language first.

Contents

1 Introduction

Project 1 asks each team to submit a complete EBNF grammar for Objectilang, an object oriented language with generics, covariance, and contravariance, described in the course handout through a small set of example programs. The *Rascal DSL Tutorial* teaches the mechanics needed to eventually turn a grammar like that into a working parser: concrete syntax rules, lexical rules, keyword exclusion, and the operators that express repetition, optionality, and choice.

Both documents assume a level of comfort with grammar design that most students have not yet practiced under time pressure. The review activity described here closes that gap. Students spend ninety minutes reconstructing, repairing, translating, and testing a grammar for a language unrelated to Objectilang, so that every mistake they are going to make in Project 1 gets made once already, in a lower stakes setting, with a partner and a teaching assistant in the room.

2 Diagnostic Analysis

2.1 What Project 1 Requires

Task 1 asks for the alphabet of terminal symbols, the set of nonterminals, the distinguished start symbol, and the production rules in EBNF, where nonterminals are written as bare words and terminals appear in quotes. The grammar has to account for object declarations, inheritance through `extends`, values and variables, methods and method calls, basic types, arithmetic, boolean, and string expressions, `if-then-else` and `for`, and generic type parameters carrying an optional covariance or contravariance modifier. Reserved words, identifiers, and the operator and punctuation alphabet are given directly in the handout.

2.2 What the Tutorial Contributes

Section 4 of the tutorial shows how an EBNF-shaped idea becomes a Rascal `syntax` rule: `'terminal'` literals, named nonterminals, and the `+`, `*`, `?`, and `|` operators, together with `lexical` rules for things like identifiers and integers, and a `keyword` declaration that excludes reserved words from the identifier alphabet. Later sections cover abstract syntax, three styles of code generation, and validation, but the review activity only needs Section 4, since Project 1 itself asks for a grammar on paper, not a running parser.

2.3 What the Skeleton Actually Contains

The folder distributed alongside the assignment, `Codigo Ordenado-2`, was inspected file by file before any activity was designed around it. It does not contain a project skeleton for Objectilang. It holds the tutorial's own reference code, organized by section, built around the tutorial's example language of Planning, PersonTasks, and Task. It has no `META-INF/RASCAL.MF`, no `src/main/rascal` layout, and no content related to Objectilang. That code was still useful: it is a fully working Rascal project in a domain other than Objectilang, exactly the kind of material the activity needed for hands on practice without handing students a solution to their actual assignment.

2.4 Anticipated Difficulties

Four difficulties show up reliably when students first design a grammar. Confusing a terminal with a nonterminal, especially when a keyword is not quoted. Choosing the wrong repetition or optionality operator, most often because the operator was inferred from what feels natural rather than from what the examples actually show. Forgetting that identifiers must exclude reserved words, an omission that produces a grammar that still accepts every positive example handed

to it, which makes the bug invisible until someone tries a case built to expose it. Representing a generic type parameter with an optional variance modifier, the hardest single rule in the whole assignment, since it requires composing an optional element with a choice inside a declaration.

3 Activity Design

3.1 Learning Objectives

By the end of the session a team should be able to tell terminals, nonterminals, the start symbol, and production rules apart in an unfamiliar grammar; turn a set of valid example programs into a complete EBNF grammar; find and justify errors in a grammar written by someone else; translate a small EBNF rule into Rascal concrete syntax; tell a `syntax` rule from a `lexical` rule and a reserved word declaration; design test cases, both positive and negative, that would expose a specific class of bug; and represent a generic type parameter with an optional variance modifier, without having seen Objectilang's own version of that rule.

3.2 Format

Ninety minutes, pairs or small groups, one computer per group already running a Rascal enabled VS Code, and a teaching assistant circulating through the room. Groups work through six missions in order, each with a time limit, up to three hints available on request, and a defined bar for clearing the mission.

3.3 Domain Choice

The main practice domain is CosmosLog, a small language for space mission logs, invented for this activity. It shares the shape of the tutorial's own Planning and Task language, one or more top level entries each containing one or more nested entries, several kinds of leaf record selected by a keyword, and an optional trailing field, but with different vocabulary and a different concrete structure, so that reconstructing its grammar is a genuine exercise rather than a transcription of something already seen.

3.4 Opening Hook: the Snake Demo

The introduction begins with a three minute demo the teaching assistant runs alone; no student needs to touch anything. A small arcade game, Rascal Snake, is already running on the instructor's machine before the room fills up: a Rascal program owns the game state and decides every move, and a browser page only draws whatever it is told to draw. Figure 1 shows that page.

The level itself, board size, wall placement, starting position, is written in a small language with its own EBNF grammar, defined in a file called `LevelSyntax.rsc`. The teaching assistant points at a few lines of a level file, points at the matching grammar rule, then breaks it live: a wall coordinate written as a word instead of a number, saved, and the editor immediately shows a parse error. Fixing it and reloading the level shows the change take effect at once, with no process restart required. The point lands in under three minutes: the grammar a team is about to write is not a paper exercise, it decides what a real program is and is not allowed to say, today for CosmosLog, in a few weeks for Objectilang.

3.5 Mission Overview

An introduction of eight minutes opens the session, including the Snake demo of Section 3.4, and a wrap up discussion of seven minutes followed by a six minute closing reflection closes it, bringing the total to ninety minutes exactly.

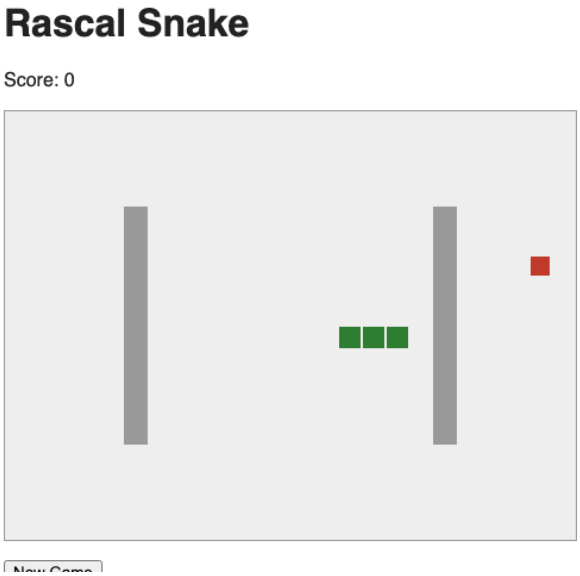

Figure 1: Rascal Snake running from a fresh level. The board, the walls, and the snake are all drawn from a state Rascal computed; the page itself contains no game logic.

Mission	Time	What it trains
1. Fast classification	6 min	Telling terminals, nonterminals, and EBNF operators apart, using a grammar the students already built
2. Grammar archaeology	15 min	Writing a complete EBNF grammar from valid example programs
3. Repair bay	13 min	Finding and correcting six planted bugs in someone else's grammar
4. Translation to the engine	15 min	Turning EBNF into Rascal concrete syntax and getting real parser feedback
5. Test case detectives	10 min	Predicting acceptance or rejection, then designing cases that reveal bugs
6. The variance challenge	10 min	Representing an optional covariance or contravariance modifier in EBNF

Table 1: The six missions, their time budget, and the skill each one trains.

4 The CosmosLog Domain-Specific Language

4.1 Reference Grammar

Listing 1 is the reference EBNF grammar for CosmosLog, the one a correct Mission 2 submission should approximate. It is never shown to students before the session; they reconstruct it themselves from the example log in Listing 3.

```
MissionControlLog ::= Mission+

Mission ::= "Mission:" ID Nave+

Nave ::= "Ship" ID "crew" INT Report+

Report ::= "Report" ReportKind "severity" ":" INT Note?

ReportKind ::= FuelReport | AnomalyReport | CommReport | DockReport

```

```
FuelReport ::= "Fuel" INT "percent"
AnomalyReport ::= "Anomaly" STRING
CommReport ::= "Comm" ID
DockReport ::= "Dock" ID

Note ::= "note" ":" STRING
```

Listing 1: CosmosLog reference grammar, EBNF form

4.2 Rascal Concrete Syntax

Listing 2 is the same grammar translated into Rascal, matching the pattern the tutorial teaches in Section 4.1. This is the grammar Mission 4 asks students to arrive at on their own, starting from a file that gives them only the lexical alphabet and a set of labeled gaps.

```
start syntax MissionControlLog
    = missionControlLog: Mission+ missions
;

syntax Mission
    = mission: 'Mission:' ID name Nave+ naves
;

syntax Nave
    = nave: 'Ship' ID name 'crew' INT crewSize Report+ reports
;

syntax Report
    = report: 'Report' ReportKind kind 'severity' ':' INT sev Note? note
;

syntax ReportKind
    = fuel: FuelReport fuelReport
    | anomaly: AnomalyReport anomalyReport
    | comm: CommReport commReport
    | dock: DockReport dockReport
;

syntax FuelReport = fuelReport: 'Fuel' INT pct 'percent' ;
syntax AnomalyReport = anomalyReport: 'Anomaly' STRING description ;
syntax CommReport = commReport: 'Comm' ID target ;
syntax DockReport = dockReport: 'Dock' ID station ;

syntax Note = note: 'note' ':' STRING text ;

keyword Reserved = "Mission" | "Ship" | "crew" | "Report" | "severity"
                 | "note" | "Fuel" | "percent" | "Anomaly" | "Comm"
                 | "Dock" | ":";
```

Listing 2: CosmosLog grammar in Rascal concrete syntax

4.3 Example Instance

```
Mission: Artemis9
Ship Falcon crew 6
Report Fuel 82 percent severity: 2
Report Comm GroundControl severity: 3

Mission: Odyssey2
Ship Odyssey crew 3
Report Dock StationAlpha severity: 1
```

```
Report Fuel 45 percent severity: 5 note: "Refuel scheduled next orbit"

Mission: Voyager3
Ship Nomad crew 12
Report Anomaly "Sensor drift on panel B" severity: 7 note: "Monitoring closely"
Ship Comet crew 2
Report Fuel 10 percent severity: 9 note: "Critical - refuel immediately"
```

Listing 3: A valid CosmosLog log, used throughout Missions 2, 4, and 5

This single log exercises every construct in the grammar: more than one mission per file, more than one ship per mission, all four report kinds, and a report with and without its optional note.

5 Mission 6 and the Objectilang Connection

Mission 6 is the deliberate bridge to the hardest part of Project 1. It asks students to design an EBNF rule for `Manifest`, a generic cargo container carrying a single type parameter with an optional covariance or contravariance modifier, in the same spirit as `List[+A]` and `Transformer[-In,+Out]` from the Project 1 handout, without ever showing them Objectilang's own grammar. One valid answer is:

```
Manifest ::= "Manifest" "[" TypeParam "]"
TypeParam ::= Variance? ID
Variance ::= "+" | "-"
```

Listing 4: A valid Mission 6 answer for the Manifest rule

Students are then asked, without writing the rule out, to explain how this would extend to two type parameters with independent modifiers, which is exactly the shape of `Transformer[-In,+Out]`. A grading rubric for this mission accepts any structurally sound answer, not only the one shown above, since the point is transferring the pattern, not matching a fixed string.

6 Materials Produced

The activity ships as a self contained folder, `cosmoslog-activity`, split into a student facing half and an instructor only half.

- `FOR_STUDENTS/student_handout.md`, the complete session handout: context, rules, all six missions, the example log, and a submission checklist.
- `FOR_STUDENTS/cosmoslog-dsl/`, a working Rascal project with the layout and lexical alphabet already filled in and the seven syntax rules left as labeled gaps for Mission 4, plus the example and test instance files used in Missions 2 and 5.
- `FOR_TA/session_script.md`, a minute by minute script covering timing, instructions, files used, and what to watch for in each mission.
- `FOR_TA/ta_guide.md`, the answer key: solutions, three tiered hints per mission, guiding questions, common mistakes, and a short formative rubric.
- `FOR_TA/reference_solutions/`, the finished grammar, the Mission 2 reference grammar, the Mission 3 bug list with evidence, the Mission 6 sample answers, and a small automated regression project described in Section 7.

A second, sibling folder, `snake-game`, holds the Rascal Snake project used for the opening demo of Section 3.4. It is not part of `cosmoslog-activity` and students never open it; it exists purely for the teaching assistant to run before the session starts, with its own setup instructions in `snake-game/README.md`.

7 Technical Integration and Verification

Every technical claim in this report was checked against a running Rascal installation rather than left as an assumption.

7.1 Compilation Check

The reference grammar was compiled with the real Rascal type checker, invoked through the `rascal-maven-plugin compile` goal. The build reported zero errors and zero undefined symbols, confirming that the grammar is valid Rascal and that `start[MissionControlLog]` resolves correctly.

7.2 Automated Regression Test

A small self contained project, `FOR_TA/reference_solutions/smoke-test/`, bundles the reference grammar together with the example log and the six Mission 5 test cases, and a `SmokeTest` module that parses each one and compares the result against its expected outcome. This is not a one time check; it can be rerun after any change to the reference grammar, or against a different Rascal distribution, in under two minutes.

Case	Expected	What it tests
Positive log	Accepted	The full example log, all four report kinds
E1	Accepted	A minimal single mission, single ship instance
E2	Accepted	Two missions, a Dock report, no note anywhere
E3	Rejected	Missing the leading `"Mission:"` literal
E4	Rejected	A string given where `severity` expects an integer
E5	Rejected	A reserved word used as a ship name
E6	Accepted	A negative crew size, syntactically legal, semantically absurd

Table 2: The seven automated test cases and their confirmed outcome.

Every one of the seven cases matched its expected outcome on the last run, across two independent execution paths: the Rascal interpreter invoked directly, and the type checker invoked through Maven. Case E6 is kept deliberately even though it passes, since it is the anchor for the closing discussion about the difference between a syntax error and a validation error, the subject of Section 8 of the tutorial.

8 Mapping Missions to Project 1

Mission	Where it resurfaces in Project 1
1	The requirement to state the sets of terminals and nonterminals explicitly
2	Turning Snippets 1 through 4 into the Objectilang grammar
3	Self review before submitting, catching the same classes of bug
4	Preparation for any future assignment that asks for a working implementation
5	Designing test cases against one's own grammar
6	`List[+A]` and `Transformer[-In,+Out]`, see Section 5

Table 3: How each mission maps back onto a concrete requirement of Project 1.

9 Conclusion

The activity turns four abstract skills, reading examples into a grammar, catching grammar bugs, translating EBNF into a working parser, and designing test cases, into something a pair of students can practice once, under time pressure, before those same skills matter for a grade. Every grammar, every test case, and every expected result in this report was verified against a running Rascal installation rather than asserted, and the automated regression project in `FOR_TA/reference_solutions/smoke-test/` lets that verification be repeated at any time.